Silver Layer (Transformation)

Sunday, 15 March 2026 11:36 AM

Uber Project: Silver Layer Transformation and One Big Table (OBT)

1. Introduction to the Silver Layer

The primary purpose of the **Silver layer** is **transformation**. It acts as a staging area where raw data from the Bronze layer is cleaned, structured, and unified. In this project, the Silver layer focuses on merging historical data (initial load) with real-time streaming data (incremental load) to create a comprehensive, queryable dataset.

2. The Challenge of Unifying Historical and Real-Time Data

In the Bronze layer, we have two distinct sources: * **`bulk_rides`**: Historical data containing potentially decades of ride records. * **`rides_raw`**: Real-time streaming data ingested from Azure Event Hubs.

Why a Simple Union is Inefficient

Performing a standard `UNION` between these two tables in every iteration (e.g., every 5 minutes) is not feasible. As the historical dataset grows into "big data," re-processing it alongside every new stream batch becomes a significant performance bottleneck.

The Solution: Streaming Append Flow

The strategy is to create a new **streaming table** where: 1. The **historical data** (`bulk_rides`) is appended once during the initial pipeline run. 2. The **streaming data** (`rides_raw`) is continuously appended as new events arrive. 3. This unified table then serves as the foundation for joining with mapping tables to create the **One Big Table (OBT)**.

3. Implementation: Building the `stg_rides` Table

This process is implemented using **Delta Live Tables (DLT)** and the `append_flow` decorator, which allows multiple data sources to write into a single target table.

3.1. Defining the Schema

To convert the raw JSON strings from the stream into a structured format, a explicit schema (`StructType`) is defined. This schema mirrors the structure of the historical `bulk_rides` table.

```
from pyspark.sql.types import *


rides_schema = StructType([
    StructField('ride_id', StringType(), True),
    StructField('confirmation_number', StringType(), True),
    StructField('passenger_id', StringType(), True),
    StructField('driver_id', StringType(), True),
    StructField('vehicle_id', StringType(), True),
    # ... (additional fields like passenger_name, driver_rating, total_fare, etc.)
    StructField('booking_timestamp', TimestampType(), True),
    StructField('total_fare', DoubleType(), True)
```

```
])
```

3.2. Creating the Staging Table

A target streaming table named `stg_rides` is created to store the structured and unified data.

```
dp.create_streaming_table("stg_rides")
```

3.3. Appending Historical Data (Bulk Flow)

The historical data is read as a stream from the `bulk_rides` table. A type cast is performed on the `booking_timestamp` to ensure it is stored as a `timestamp` rather than a `string`, enabling time-based analytics and window functions.

```
@dp.append_flow(target = "stg_rides")

def rides_bulk():

    df = spark.readStream.table("bulk_rides")

    df = df.withColumn("booking_timestamp", col("booking_timestamp").cast("timestamp"))

    return df
```

3.4. Appending Real-Time Data (Streaming Flow)

The real-time data is read from `rides_raw`. Since the raw data is in JSON string format, the `from_json` function is used along with the `rides_schema` to parse the JSON into a structured format.

```
@dp.append_flow(target = "stg_rides")

def rides_stream():

    df = spark.readStream.table("rides_raw")

    # Parse JSON string to structured format and expand columns

    df_parsed = df.withColumn("parsed_rides", from_json(col("rides"), rides_schema))\

                .select("parsed_rides.*")

    return df_parsed
```

4. Data Normalization and Cleaning

Raw events often arrive in unstructured formats. For example, a single `rides` column might contain a complex JSON string: `{"ride_id": "10812...", "passenger_name": "Willie Powell", ...}`

The use of `from_json()` and `.select("parsed_rides.*")` effectively "flattens" this JSON string into individual, typed columns. This normalization ensures that the data in `stg_rides` is clean, consistent, and ready for analytical queries.

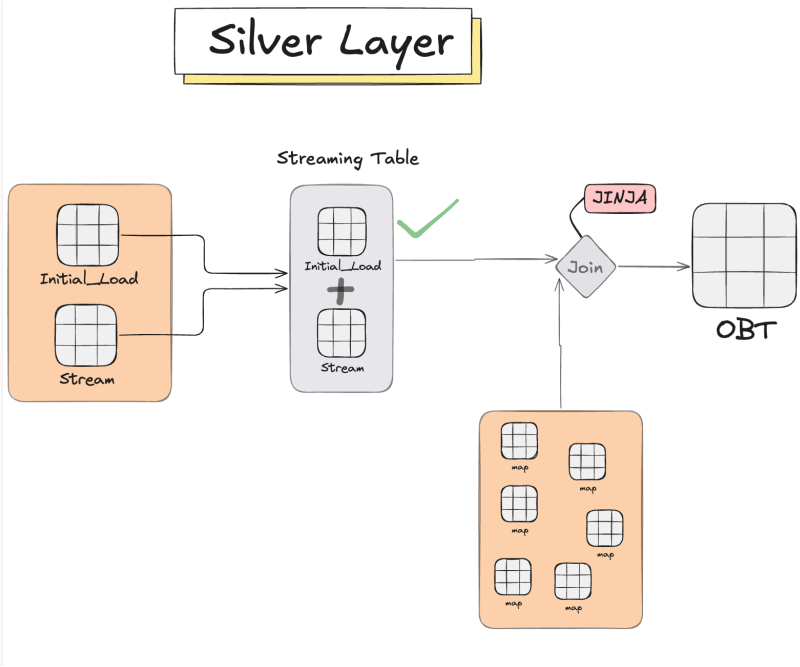
5. Verification and Incremental Processing

Once the pipeline is executed: * **Initial Run**: The `stg_rides` table will contain the union of historical records (e.g., 2,000+ records) and any existing stream data. * **Subsequent Runs**: Because DLT uses **incremental processing**, running the pipeline again when no new data has arrived will result in 0 new records being added. * **New Data**: If a new record is added via the `api.py` producer, only that specific new record will be processed and appended to `stg_rides`.

6. Architecture Evolution

The data pipeline architecture has now evolved to:

```
Uber Ride App → Azure Event Hub → rides_raw (Bronze) → stg_rides (Silver)
```

The `stg_rides` table in the Silver layer now serves as the high-quality, structured staging area for the

final transition to the Gold layer.